



Facultad de Ingeniería

Proyecto Final: Creación de página web de telemedicina y consultas médicas para la empresa “Medilink”

Curso: Herramientas de Desarrollo
Sección: 23280 — Grupo 6

Docente:

Milla Flores, José Luis

Integrantes:

Antiquera Farfán, Ronaldo Isaac
Campos Toribio, Liliana Lilibeth
Adrián Ore, Luis Francisco

Lima, Perú
2026 – I

Índice

Introducción	4
Contexto Actual.....	4
Planteamiento del Problema	5
Problemática.....	5
Problema General	5
Problemas Específicos	6
Objetivos	6
Objetivo General	6
Objetivos Específicos	6
Justificación	7
Diagramas del Sistema	8
Diagrama Conceptual.....	8
Diagrama de Entidad-Relación	9
Diagrama Físico	9
Marco Teórico	10
Antecedentes de la Investigación	10
<i>Antecedentes Internacionales.....</i>	<i>10</i>
<i>Antecedentes Nacionales</i>	<i>10</i>
Bases Teóricas	11
<i>Teoría de la Transformación Digital.....</i>	<i>11</i>
<i>Teoría de la Telemedicina.....</i>	<i>11</i>
<i>Arquitectura de Software Escalable</i>	<i>11</i>
<i>Metodologías Ágiles de Desarrollo de Software</i>	<i>12</i>
<i>Seguridad de la Información</i>	<i>12</i>
<i>Bases de Datos Relacionales.....</i>	<i>12</i>
Marco Conceptual.....	13
Relación del Marco Conceptual con la Propuesta	13
Hipótesis.....	14
Hipótesis General	14
Hipótesis Específicas.....	14
Metodología	14
Justificación de la Metodología Scrum	15
Artefactos de Scrum en el Proyecto MediLink	15

Cronograma de Sprints	16
<i>Sprint 0: Configuración del Proyecto (Semana 1-2)</i>	<i>16</i>
<i>Sprint 1: Autenticación y Seguridad (Semana 3-4)</i>	<i>16</i>
<i>Sprint 2: Gestión de Citas y Paneles (Semana 5-6)</i>	<i>16</i>
Diagrama de Gantt	17
Herramientas de Desarrollo Utilizadas.....	17
Desarrollo e Implementación del Proyecto.....	19
Control de Versiones y Gestión de Ramas.....	19
Gestión Ágil e Implementación de Controladores REST.....	22
Resolución de Conflictos de Git.....	25
Integración Continua (CI) con GitHub Actions.....	27
Calidad de Código con SonarCloud	29
Cobertura de Pruebas con JaCoCo	33
Contenedores con Docker.....	34
Despliegue en la Nube con Railway	36
Resultados.....	38
Conclusiones	40
Recomendaciones	41
Referencias.....	42

Introducción

La empresa MediLink es una clínica dedicada a brindar servicios de atención médica a sus pacientes. Sin embargo, actualmente enfrenta dificultades en la gestión de sus consultas, porque el registro de estas se realiza de manera manual. Este método provoca varios problemas como la pérdida de información, duplicidad de datos, retrasos en la atención y problemas al momento de realizar un seguimiento a un paciente. La falta de un sistema digital eficiente no solo afecta la administración interna, sino también la calidad del servicio que reciben los pacientes, generando insatisfacción y limitando la capacidad de respuesta de la clínica frente a una creciente demanda de atención médica.

En un contexto global, las plataformas de telemedicina se han consolidado como alternativas para brindar atención médica a distancia y facilitar la reserva de citas y las consultas en tiempo real. En este sentido, MediLink busca transformar su modelo de atención mediante una solución tecnológica que supere las limitaciones de los procesos manuales (Orosco et al., 2021).

Contexto Actual

En el Perú, la disponibilidad de internet constituye un factor relevante para la implementación de plataformas de telemedicina. Durante el primer trimestre de 2024, el 77,7 % de la población de seis años a más utilizó internet; sin embargo, persistió una brecha territorial, pues el acceso alcanzó el 88,5 % en Lima Metropolitana y el 48,5 % en el área rural (Instituto Nacional de Estadística e Informática [INEI], 2024).

Planteamiento del Problema

Problemática

La gestión manual de citas e historias clínicas limita la eficiencia de la atención y dificulta el seguimiento de los pacientes. A esta situación se suma la brecha de conectividad existente entre las zonas urbanas y rurales, la cual debe considerarse al diseñar una plataforma de telemedicina accesible y adaptable a diferentes condiciones de conexión (INEI, 2024).

Esta situación no solo limita la oportunidad de atención, sino que también ocasiona efectos graves, como diagnósticos que llegan tarde, interrupción de tratamientos y una menor capacidad de controlar enfermedades crónicas. A ello se suma la concentración de profesionales de la salud en las principales ciudades, lo que intensifica la saturación de los hospitales urbanos y deja a muchas provincias sin especialistas suficientes.

Ante esta realidad, surge la necesidad de explorar soluciones innovadoras que trasciendan las limitaciones físicas del sistema sanitario. Una alternativa viable es MediLink, una plataforma de telemedicina orientada a ampliar el acceso a consultas médicas seguras y oportunas, contribuyendo a cerrar la brecha entre el ámbito urbano y rural.

Problema General

¿Cómo puede la clínica MediLink desarrollar e implementar una plataforma web moderna y funcional que permita ofrecer consultas y atención médica eficiente en línea?

Problemas Específicos

- ¿Qué arquitectura de backend (utilizando tecnologías como Spring Boot) es la más adecuada para asegurar la escalabilidad, el alto rendimiento y el soporte de una alta concurrencia de usuarios en la nueva plataforma de telemedicina?
- ¿Cómo se debe diseñar e integrar un canal de comunicación de telemedicina dedicado (basado en WebRTC) que garantice la calidad de la videollamada y la interacción fluida necesaria para el diagnóstico y el seguimiento médico remoto?
- ¿Qué mecanismos robustos de seguridad digital deben implementarse para mitigar el alto riesgo de vulnerabilidad y asegurar la confidencialidad, integridad y protección de los datos clínicos de los pacientes?

Objetivos

Objetivo General

Desarrollar una plataforma web de telemedicina que permita realizar consultas médicas en tiempo real, gestionar datos clínicos y garantizar la confidencialidad de la información.

Objetivos Específicos

- Implementar una arquitectura web moderna utilizando Spring Boot y Spring Web, con optimización del código y automatización de procesos dentro del entorno backend, con el fin de asegurar un sistema escalable.
- Desarrollar el módulo de atención médica en tiempo real implementando WebRTC para videollamadas, y aplicando interfaces dinámicas con HTML, CSS, JavaScript y

componentes visuales como Bootstrap, permitiendo la comunicación fluida entre médicos y pacientes.

- Operacionalizar mecanismos robustos de seguridad digital mediante Spring Security, autenticación basada en JWT, validaciones con Hibernate Validator y control de roles y permisos, garantizando la confidencialidad, integridad y protección de los datos clínicos almacenados en la plataforma.

Justificación

La implementación de una plataforma web de telemedicina es una respuesta estratégica a las limitaciones tecnológicas y las demandas actuales del sector salud. La clínica enfrenta barreras significativas, incluyendo la discontinuidad del tratamiento, la vulnerabilidad en la seguridad de los datos clínicos y la lentitud en la comunicación médico-paciente. Este proyecto no solo resuelve estos problemas, sino que también posiciona a la institución en la vanguardia de la transformación digital sanitaria.

El proyecto se sustenta en una arquitectura robusta y moderna, utilizando Spring Boot para construir un backend modular, escalable y optimizado, que facilita el mantenimiento y la automatización de procesos. Las interfaces dinámicas, desarrolladas con HTML, CSS, JavaScript y Bootstrap, aseguran una experiencia de usuario intuitiva y adaptable, crucial para la accesibilidad de los pacientes y la eficiencia de los profesionales de la salud.

La seguridad digital es crítica y se aborda mediante Spring Security, JWT (JSON Web Tokens) y Hibernate Validator. Esta combinación garantiza la protección de datos sensibles, una

autenticación segura y una gestión rigurosa de roles y permisos, asegurando la integridad y privacidad de la información clínica.

Diagramas del Sistema

Diagrama Conceptual

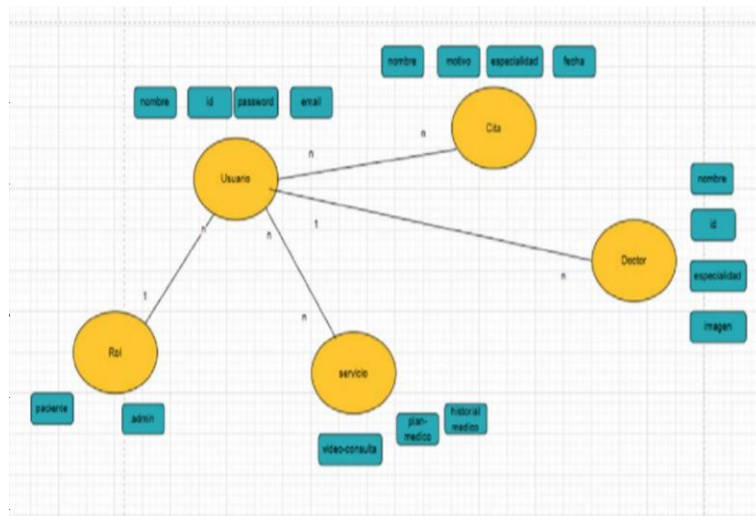


Figura 1. Diagrama conceptual del sistema MediLink.

Diagrama de Entidad-Relación

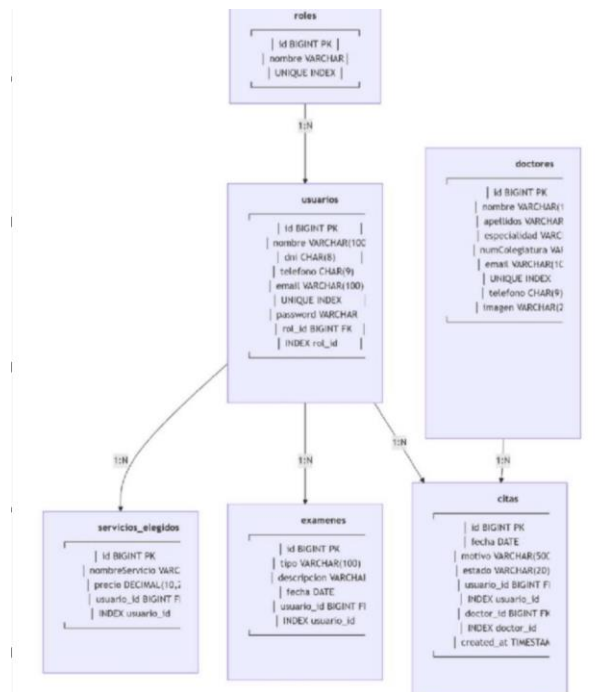


Figura 2. Modelo entidad-relación de la base de datos.

Diagrama Físico

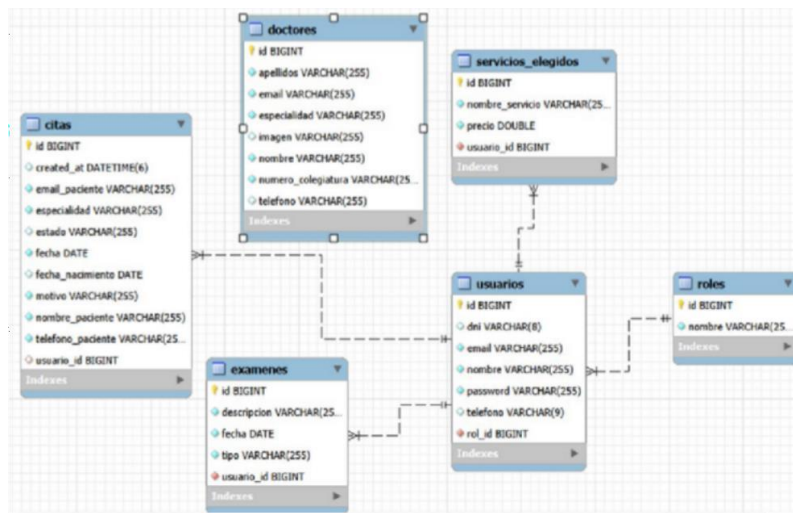


Figura 3. Diagrama físico de la base de datos.

Marco Teórico

Antecedentes de la Investigación

Antecedentes Internacionales

Título: Desarrollo de una aplicación web de telemedicina en beneficio de la comunidad de la Universidad Técnica Estatal de Quevedo (Orosco et al., 2021).

Objetivo: Desarrollar una web app de telemedicina con el uso de herramientas libres para dar soporte al departamento médico de la UTEQ.

Método de análisis: SUS (System Usability Scale).

Técnicas empleadas: WordPress, Bookly, SnatchBot, Buddy Meet y Jitsi Meet.

Resultados: se logró ayudar a los médicos a realizar consultas de manera remota y se incrementó el número de personas que accedían a consultas médicas con mayor frecuencia.

Antecedentes Nacionales

Título: Implementación de software para citas médicas a través de canales virtuales (Padilla et al., 2024).

Objetivos: controlar o prevenir enfermedades, brindar atención desde los hogares, mantener una base de datos con el expediente médico del usuario y posicionar el software de telemedicina en el mercado peruano.

Método de análisis: Kanban.

Técnicas empleadas: Frontend, Java Spring Boot, Spring MVC, MySQL, Flutter, Zoom API y Spring Security.

Resultados: atención médica 24/7, mejora en el procedimiento de agendamiento de citas y mayor acceso a servicios de salud.

Bases Teóricas

Teoría de la Transformación Digital

La transformación digital implica incorporar tecnologías informáticas para modificar procesos, formas de trabajo y modelos de prestación de servicios. La adopción masiva de estas tecnologías transforma la relación entre las organizaciones, el trabajo y los usuarios (Beas, 2014). En MediLink, este enfoque sustenta la migración de los registros manuales hacia una plataforma web orientada a la gestión de citas y consultas médicas.

Teoría de la Telemedicina

La telemedicina utiliza herramientas digitales para brindar atención médica a distancia. Las aplicaciones web de este tipo pueden incorporar reserva de citas, asistencia virtual y acceso a consultas en tiempo real, ampliando las posibilidades de comunicación entre pacientes y profesionales de la salud (Orosco et al., 2021). En MediLink, estos principios orientan el desarrollo de una plataforma accesible para la atención y el seguimiento médico.

Arquitectura de Software Escalable

MediLink adopta una arquitectura en capas conformada por controladores, servicios y repositorios. Esta separación de responsabilidades facilita el mantenimiento del código, la

ejecución de pruebas y la incorporación progresiva de nuevas funcionalidades, como la gestión de usuarios, médicos, especialidades y citas.

Metodologías Ágiles de Desarrollo de Software

Para organizar el desarrollo de MediLink se utilizó una metodología ágil, distribuyendo las actividades en sprints y gestionando el trabajo mediante GitHub Projects, Issues y un tablero Kanban. El uso de canales virtuales y una organización incremental también ha sido aplicado en propuestas recientes de software para citas médicas (Padilla et al., 2024).

Seguridad de la Información

La seguridad informática comprende prácticas orientadas a proteger los sistemas y la información frente a accesos, modificaciones o usos no autorizados. Sus principios principales incluyen la confidencialidad, integridad, disponibilidad y autenticación (Universidad de Cataluña, s. f.). En MediLink, estos principios se aplican mediante Spring Security, JWT, control de roles, validaciones y protección de los datos clínicos.

Bases de Datos Relacionales

MediLink utiliza MySQL para organizar la información en tablas relacionadas mediante claves primarias y foráneas. Esta estructura permite mantener la consistencia de los datos correspondientes a usuarios, roles, médicos, citas y especialidades, además de reducir la redundancia mediante una adecuada normalización de la base de datos.

Marco Conceptual

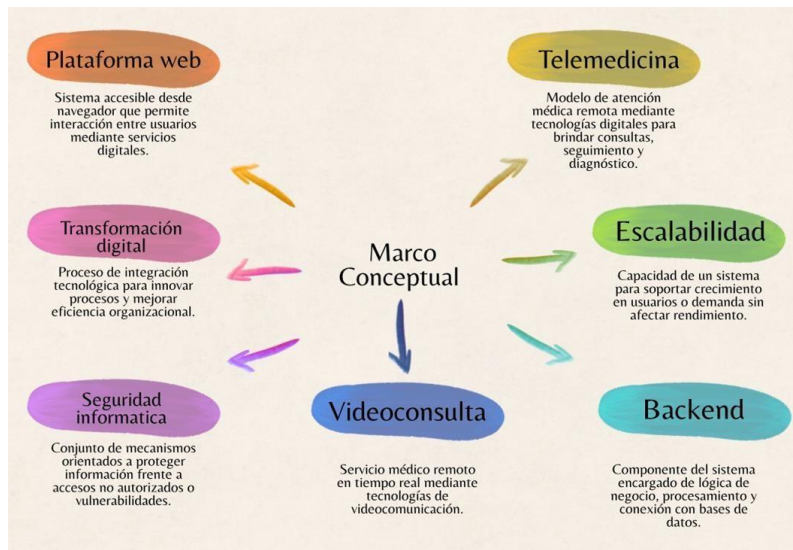


Figura 4. Mapa del marco conceptual del proyecto.

Relación del Marco Conceptual con la Propuesta

Las teorías presentadas fundamentan la propuesta MediLink al demostrar que la transformación digital, la telemedicina, la gestión por procesos, las arquitecturas escalables y la seguridad informática constituyen bases conceptuales y tecnológicas para diseñar una plataforma web moderna de atención médica. El proyecto integra dichos fundamentos para ofrecer una solución innovadora, segura y funcional que contribuya a mejorar el acceso a servicios médicos y a optimizar la gestión clínica.

Hipótesis

Hipótesis General

La implementación de una plataforma web de telemedicina para MediLink mejorará significativamente la eficiencia en la atención médica, optimizará la gestión de consultas y garantizará mayor seguridad en el manejo de información clínica.

Hipótesis Específicas

- La implementación de una arquitectura basada en Spring Boot permitirá mejorar la escalabilidad y el rendimiento de la plataforma de telemedicina.
- La integración de un canal de comunicación en tiempo real mediante WebRTC mejorará la interacción entre médicos y pacientes, facilitando consultas remotas eficientes.
- La aplicación de mecanismos de seguridad como Spring Security y autenticación JWT fortalecerá la confidencialidad, integridad y protección de los datos clínicos.

Metodología

Para el desarrollo de la plataforma web de telemedicina MediLink se seleccionó la metodología ágil Scrum. Esta decisión se fundamenta en la necesidad de construir un sistema complejo, con altos requerimientos de seguridad y una experiencia de usuario crítica, donde la flexibilidad, la entrega incremental y la adaptación al cambio son factores clave para el éxito del proyecto.

Justificación de la Metodología Scrum

Tabla 1

Relación entre las necesidades de MediLink y los beneficios de Scrum

Necesidad del Proyecto MediLink	Beneficio de Scrum
Requisitos cambiantes (nuevas necesidades de médicos/pacientes)	Los sprints cortos permiten adaptar el backlog y priorizar nuevas funcionalidades cada 2-4 semanas.
Alta calidad y seguridad (datos clínicos sensibles)	La integración continua y las revisiones al final de cada sprint aseguran que se cumplan los estándares de calidad y seguridad.
Colaboración constante (equipo de 3 integrantes)	Scrum fomenta la comunicación diaria y la auto-organización, maximizando la eficiencia del equipo pequeño.
Feedback temprano (del docente / cliente potencial)	Las revisiones de sprint (Sprint Review) permiten mostrar el producto funcional y recibir retroalimentación para el siguiente ciclo.

Artefactos de Scrum en el Proyecto MediLink

- **Product Backlog:** lista ordenada de todas las funcionalidades, mejoras y correcciones necesarias para MediLink, gestionada como un proyecto en GitHub Projects, donde cada Issue representa una historia de usuario etiquetada por tipo (feature, bug, security) y por objetivo específico.
- **Sprint Backlog:** conjunto de Issues del Product Backlog que el equipo se compromete a completar durante un Sprint, representado en GitHub Projects moviendo las tarjetas a la columna “Sprint Actual” o “In Progress”.

- Incremento: suma de todas las tareas completadas durante un Sprint, convertida en una funcionalidad operativa y desplegable; en cada incremento, MediLink debe ser un producto más completo y estable que al inicio del Sprint.

Cronograma de Sprints

Sprint 0: Configuración del Proyecto (Semana 1-2)

Objetivo: tener el proyecto Spring Boot funcionando localmente con la base de datos conectada. Se configuró Spring Initializr con las dependencias Spring Web, Spring Data JPA, Thymeleaf, Lombok y Validation; se crearon las ramas main y develop protegidas, el .gitignore para Java/Spring Boot y la licencia del repositorio; se conectó la base de datos (MySQL/H2) y se crearon las primeras entidades (Usuario, Rol) con sus repositorios.

Sprint 1: Autenticación y Seguridad (Semana 3-4)

Se agregó la dependencia de Spring Security y se creó SecurityConfig.java con los filtros JWT correspondientes (JwtUtil y JwtAuthenticationFilter). Se implementaron los DTOs de login y registro, los endpoints /api/auth/login, /api/auth/registro y /api/auth/logout, el cifrado de contraseñas con BCryptPasswordEncoder y el control de roles mediante anotaciones @PreAuthorize.

Sprint 2: Gestión de Citas y Paneles (Semana 5-6)

Objetivo: implementar el CRUD de citas y los paneles para médicos y pacientes. Se crearon las entidades Cita, Especialidad y HorarioDisponible con sus repositorios (findByPaciente, findByMedico, findByFecha), el servicio CitaService (agendado, cancelación y validación de disponibilidad), el CitaController con los endpoints CRUD, los controladores de paciente y

médico, las vistas Thymeleaf (citasUsuario.html, panelMedico.html, historialPaciente.html) y el buscador de médicos por especialidad y horario disponible.

Diagrama de Gantt

El siguiente diagrama de Gantt muestra el cronograma de construcción del proyecto y cómo los integrantes fueron incorporando componentes a lo largo de las distintas fases.

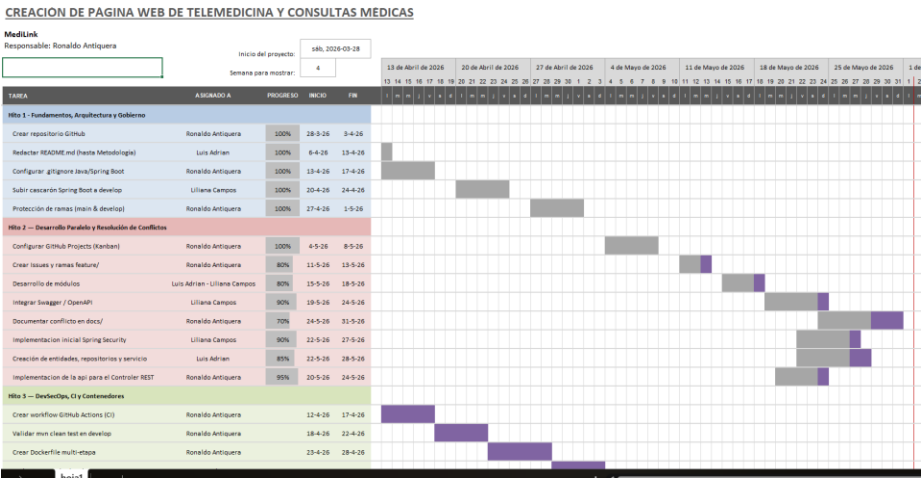


Figura 5. Cronograma general del proyecto (Hito 1).

Herramientas de Desarrollo Utilizadas

A lo largo del proyecto se utilizó un conjunto de herramientas y tecnologías para cubrir las distintas etapas del ciclo de vida del software: diseño, programación, control de versiones, pruebas, calidad de código, despliegue y comunicación del equipo. La siguiente tabla resume cada herramienta, su categoría y el uso específico que se le dio dentro de MediLink.

Tabla 2*Herramientas de desarrollo utilizadas en el proyecto MediLink*

Herramienta	Categoría	¿Para qué se usó en MediLink?
Spring Boot	Backend	Framework principal usado para construir la API REST del backend: controladores, servicios, repositorios y seguridad.
MySQL	Base de datos	Motor de base de datos relacional donde se almacenan usuarios, citas, especialidades e historiales clínicos.
HTML5 / CSS3 / JavaScript	Frontend	Lenguajes base para construir las vistas (Thymeleaf) y la interactividad del lado del cliente.
Bootstrap	Frontend / UI	Librería de componentes usada para maquetar formularios y paneles de manera responsiva.
Materialize CSS	Frontend / UI	Framework de estilos usado en algunas vistas para dar una apariencia moderna tipo Material Design.
Tailwind CSS	Frontend / UI	Utilizado para prototipar y ajustar estilos de forma rápida mediante clases utilitarias.
Figma	Diseño	Herramienta para diseñar los mockups y prototipos de las pantallas antes de programarlas.
Visual Studio Code	IDE	Editor de código usado principalmente para el frontend, la configuración de Docker y los workflows de GitHub Actions.
IntelliJ IDEA	IDE	IDE usado para el desarrollo del backend en Java/Spring Boot (autocompletado, debugging y refactorización).
GitHub	Control de versiones	Repositorio remoto del proyecto; gestión de ramas protegidas, Pull Requests, Issues y GitHub Projects (Kanban).
GitHub Actions	CI/CD	Automatiza la compilación y pruebas (mvn clean test) en cada push, y ejecuta el análisis de SonarCloud.
Postman	Pruebas de API	Usado para probar manualmente los endpoints REST (login, registro, CRUD de citas) durante el desarrollo.
Docker	Contenedores	Empaquetado de la aplicación y la base de datos en imágenes mediante un Dockerfile multi-etapa y docker-compose.
SonarCloud	Calidad de código	Análisis estático que detecta bugs, vulnerabilidades y duplicidad de código en cada integración.

Herramienta	Categoría	¿Para qué se usó en MediLink?
Railway	Despliegue (Cloud)	Plataforma PaaS donde se desplegó la aplicación en producción, sin gestionar servidores manualmente.
JWT (JSON Web Token)	Seguridad	Estándar usado junto a Spring Security para la autenticación y autorización basada en tokens.
Zoom	Comunicación	Reuniones semanales del equipo para coordinar sprints y revisar avances con el docente.
WhatsApp	Comunicación	Canal de comunicación diaria del equipo para coordinación rápida y envío de evidencias.
Claude (Anthropic)	Inteligencia Artificial	Apoyo en la redacción y ordenamiento del informe, revisión de código y generación de documentación.
ChatGPT (OpenAI)	Inteligencia Artificial	Apoyo en la resolución de dudas técnicas puntuales y generación de fragmentos de código de referencia.

Desarrollo e Implementación del Proyecto

Control de Versiones y Gestión de Ramas

Evidencias de protección y revisión del código

Las ramas principales se protegieron para impedir cambios directos y exigir la revisión del equipo antes de integrar nuevas funcionalidades. Las evidencias muestran las reglas aplicadas en main y develop, así como la revisión y aprobación de cambios mediante Pull Requests.

Rama main

Code and automation

- Branches**
- Tags
- Rules
- Actions
- Models Preview
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security
- Deploy keys
- Secrets and variables

Branch name pattern *

Applies to 1 branch

main

Protect matching branches

☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☒ **Require approvals**
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

Required number of approvals before merging: 2

Rama develop

Patrón de nombre de rama *

Se aplica a 1 sucursal

desarrollar

Proteja las ramas coincidentes

☒ **Se requiere una solicitud de extracción antes de la fusión.**
Cuando está habilitada, todas las confirmaciones deben realizarse en una rama no protegida y enviarse mediante una solicitud de extracción antes de que puedan fusionarse en una rama que cumpla con esta regla.

☒ **Requiere aprobaciones**
Cuando está habilitada, las solicitudes de extracción dirigidas a una rama coincidente requieren una serie de aprobaciones y no se deben solicitar cambios antes de que puedan fusionarse.

Número de aprobaciones necesarias antes de la fusión: 2

Figura 6. Configuración de protección de las ramas main y develop.

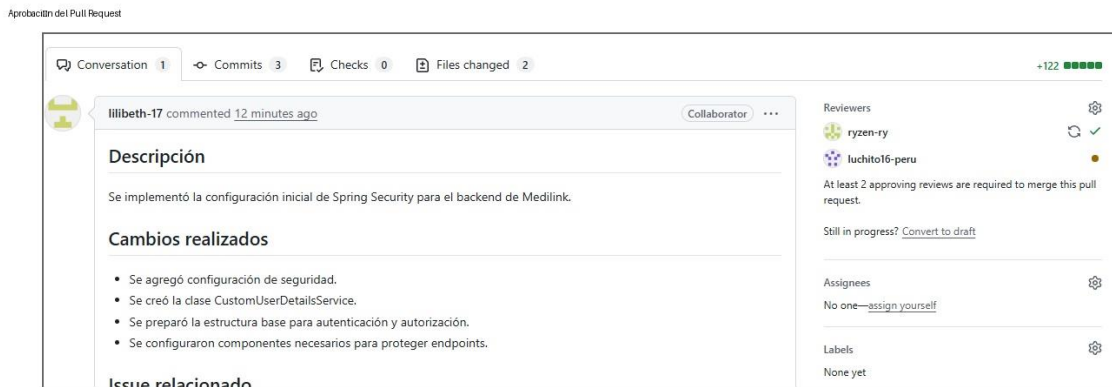
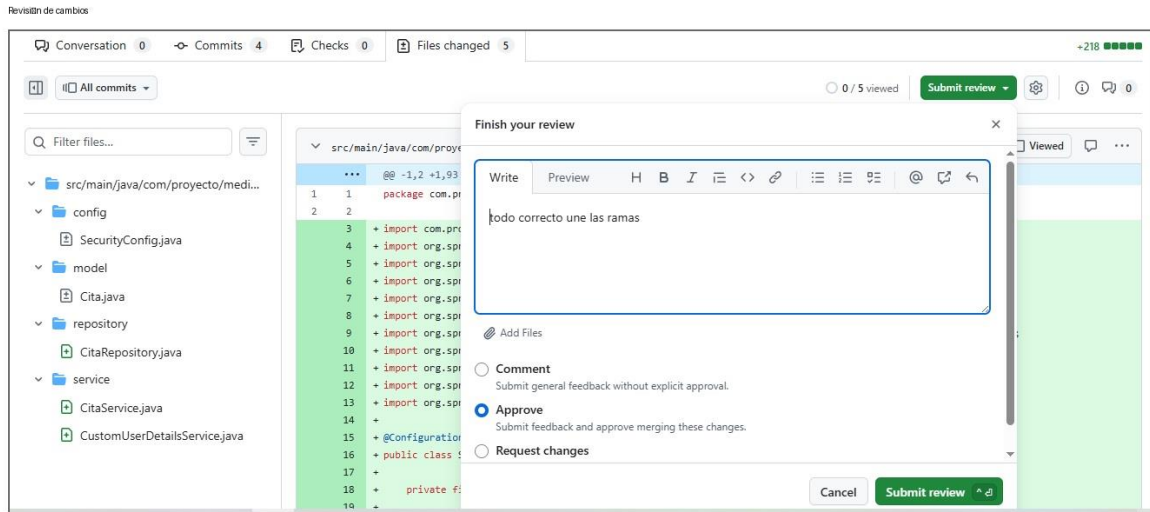


Figura 7. Revisión y aprobación de cambios mediante Pull Request.

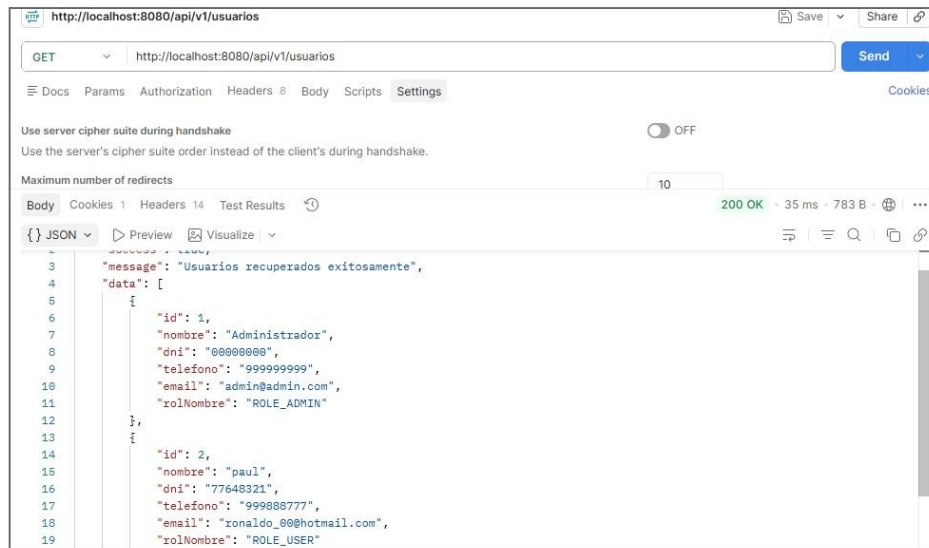
Se configuraron reglas de protección tanto para la rama main como para develop, exigiendo un mínimo de dos revisiones aprobadas antes de poder fusionar un Pull Request. Esta política obliga al equipo a realizar revisiones de código (Code Review) constantes y evita que se suban cambios directamente a las ramas principales sin validación.

Gestión Ágil e Implementación de Controladores REST

Pruebas funcionales y gestión ágil

Los endpoints y las principales operaciones del sistema fueron comprobados durante el desarrollo. Asimismo, el trabajo del equipo se organizó mediante un tablero Kanban que permitió visualizar tareas pendientes, actividades en progreso y trabajos completados.

Prueba del endpoint



Registro desde la interfaz

Registrar Nuevo Doctor

Nombre

Apellidos

Especialidad

Número de Colegiatura

Email

Teléfono

Imagen del Doctor

Doctores Registrados

Nombre	Especialidad	Colegiatura	Email	Teléfono	Acciones
 isacc ishaaa	pediatra	00000293	countercito10@gmail.com	997076185	Eliminar

Listado de médicos

Doctores Registrados					
Nombre	Especialidad	Colegiatura	Email	Teléfono	Acciones
 isacc ishaaa	pediatra	00000293	countercito10@gmail.com	997076185	Eliminar
 adriana ramos	neurologia	023300293	adriana@gmail.com	997862231	Eliminar

Figura 8. Pruebas de controladores REST y funcionamiento del módulo de médicos.

Title	Assignees	Status	Linked pull requests	Sub-i
1 subir informe final del proyecto en pdf #9	luchito16-peru	In Progress		
2 configuracion del spring security #20	lilibeth-17	Done		
3 añadir la entidad doctor ,reportorio, servicio #27	luchito16-peru	Done		
4 añadir la entidad cita #34	lilibeth-17	Done	#44	
5 subir el model examen #35	luchito16-peru	Done		
6 implementar los controladores de las vistas publicas #28	ryzen-ry	Done		
7 implementar controladores REST #29	ryzen-ry	Done		
8 configuracion del usuario admin por defecto #21	ryzen-ry	Done		

Figura 9. Tablero Kanban utilizado para organizar las tareas del proyecto.

Documentación de la API con Swagger/OpenAPI

Swagger/OpenAPI permitió visualizar y probar los endpoints disponibles, sus métodos HTTP, parámetros y respuestas esperadas desde una interfaz web.

```

1  # Resumen: Integración de Swagger/OpenAPI en Medilink
9  ## Cambios Realizados

33 ### 4. **Controladores REST documentados** ✓
34
35 #### **AuthRestController**
36 - `@Tag(name = "Autenticación")`
37 - `POST /api/v1/auth/login` - Iniciar sesión
38 - `POST /api/v1/auth/logout` - Cerrar sesión
39 - `GET /api/v1/auth/me` - Obtener usuario autenticado
40 - Cada método con `@Operation` y `@ApiResponse`
41
42 #### **UsuarioRestController**
43 - `@Tag(name = "Usuarios")`
44 - `GET /api/v1/usuarios` - Obtener todos los usuarios
45 - `GET /api/v1/usuarios/{id}` - Obtener usuario por ID
46 - `POST /api/v1/usuarios/registro` - Registrar nuevo usuario
47 - `PUT /api/v1/usuarios/{id}` - Actualizar usuario
48 - `DELETE /api/v1/usuarios/{id}` - Eliminar usuario
49 - Todos con documentación de parámetros y respuestas
50
51 #### **DoctorRestController**
52 - `@Tag(name = "Doctores")`
53 - `GET /api/v1/doctores` - Obtener todos los doctores

```

Figura 10. Documentación automática de la API REST mediante Swagger/OpenAPI.

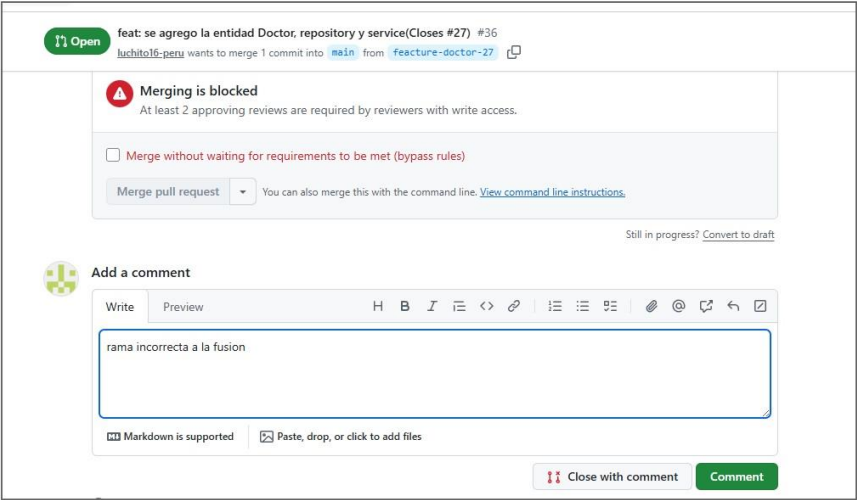
El trabajo se organizó mediante un tablero Kanban en GitHub Projects, donde cada Issue se vincula a una rama feature/ y se cierra automáticamente al fusionar el Pull Request correspondiente. Los controladores REST fueron probados manualmente con Postman durante el desarrollo, verificando operaciones como la obtención de usuarios registrados y la creación de nuevos doctores dentro del sistema.

Resolución de Conflictos de Git

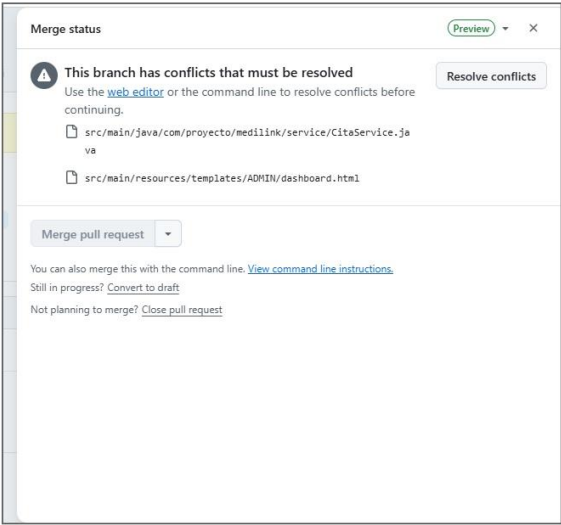
Evidencias de resolución del conflicto

Durante la integración de ramas se identificó un conflicto ocasionado por modificaciones simultáneas. El equipo comparó los cambios, resolvió las diferencias en Visual Studio Code y documentó el procedimiento dentro del repositorio.

Error de integración



Resolución documentada



Comparación y corrección en VS Code

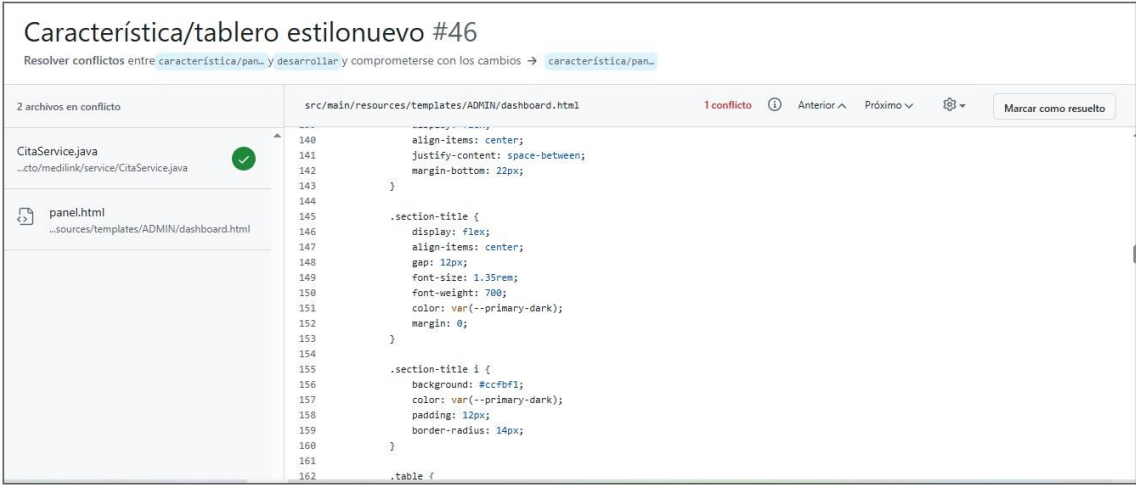


Figura 11. Identificación, resolución y documentación de un conflicto de Git.

Durante el desarrollo paralelo del equipo se presentó un conflicto de fusión (merge conflict) en dos archivos del proyecto. El conflicto fue identificado, analizado y resuelto utilizando Visual Studio Code como herramienta de comparación de cambios, documentando en el repositorio los archivos afectados y el procedimiento seguido para su resolución.

Integración Continua (CI) con GitHub Actions

Evidencias del flujo de integración continua

El workflow automatizado ejecutó la preparación del entorno, la compilación y las pruebas del proyecto. Las capturas muestran la configuración del flujo y sus ejecuciones dentro del repositorio.

Workflow configurado

Actions

All workflows

Automatic Dependency Submission

Java CI with Maven

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

new workflow

maven.yml

6 workflow runs

	Event	Status	Branch	Actor
Update maven.yml	Java CI with Maven #6: Commit 317cc5d pushed by ryzen-ry	main	6 minutes ago	...
Update maven.yml	Java CI with Maven #5: Commit 0718c4 pushed by ryzen-ry	main	11 minutes ago	...
Update maven.yml	Java CI with Maven #4: Commit d305555 pushed by ryzen-ry	main	20 minutes ago	...
Update application.properties	Java CI with Maven #3: Commit 68323cd pushed by ryzen-ry	main	24 minutes ago	...
Update maven.yml	Java CI with Maven #2: Commit e4bd90a pushed by ryzen-ry	main	30 minutes ago	...
Create maven.yml	Java CI with Maven #1: Commit 7387144 pushed by ryzen-ry	main	45 minutes ago	...

Ejecución del flujo

General

Acceso

Colaboradores

Opciones de moderación

Código y automatización

Sucursales

Etiquetas

Normas

Comportamiento

Webhooks

Copiloto

Entornos

Espacios de código

Seguridad avanzada

Las funciones de seguridad avanzadas ayudan a mantener su repositorio seguro y actualizado. Al habilitar estas funciones, nos otorga permiso para realizar análisis de solo lectura en su repositorio.

Informes privados sobre vulnerabilidades

Permita que su comunidad reporte de forma privada posibles vulnerabilidades de seguridad a los mantenedores y propietarios de repositorios. [Obtenga más información sobre cómo reportar vulnerabilidades de forma privada.](#)

Permitir

Grafo de dependencias

Comprenda sus dependencias.

Desactivar

Envío automático de dependencias

Detecta e informa automáticamente sobre las dependencias en tiempo de compilación para ecosistemas seleccionados.

Activado

Dependabot

Mantén tus dependencias seguras y actualizadas. [Obtén más información sobre Dependabot.](#)

Archivo de configuración

```

8
9 name: Java CI with Maven
10
11 on:
12   push:
13     branches: [ "develop", "main" ]
14   pull_request:
15     branches: [ "develop", "main" ]
16
17 jobs:
18   build:
19     permissions:
20       contents: write # Para leer el código y el pom.xml
21       security-events: write # <--- ESTE ES EL QUE LE FALTA para subir el reporte
22
23     runs-on: ubuntu-latest
24
25     services:
26       mysql:
27         image: mysql:8.0
28         env:
29           MYSQL_ROOT_PASSWORD: 12345678
30           MYSQL_DATABASE: prueba
31         ports:
32           - 3306:3306
33         options: --health-cmd="mysqladmin ping" --health-interval=10s --health-timeout=5s --health-retries=3
34

```

Dependabot

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security

Files

main

Go to file

.github/workflows

maven.yml

sonarcloud.yml

.mvn

docs

informes

resultados

Medilink / .github / workflows / maven.yml

ryzen-ry Update maven.yml

Code Blame 46 lines (39 loc) · 1.44 KB

1

2

3

4

5

6

7

8

9

10

Figura 12. Configuración y ejecución del flujo de GitHub Actions.

Para esta entrega se configuró la integración continua (CI) mediante GitHub Actions. Cada vez que se sube código (git push) o se abre un Pull Request hacia las ramas develop o main, un servidor en la nube de GitHub se activa y ejecuta automáticamente el ciclo de compilación y pruebas de la aplicación (mvn clean test), validando el proyecto contra una base de datos MySQL de prueba con las variables de entorno configuradas.

El flujo principal de CI ejecuta, en orden: el checkout del código, la configuración del JDK 17 (Temurin), la compilación con Maven y la actualización del grafo de dependencias, mientras un servicio de MySQL corre en paralelo para las pruebas de integración.

Calidad de Código con SonarCloud

Resultados del análisis estático

SonarCloud se utilizó para identificar problemas de mantenibilidad, duplicación, bugs y vulnerabilidades. Las evidencias permiten comparar el estado inicial con los resultados obtenidos después de atender las observaciones detectadas.

Primer análisis

Quality Gate

Resultados tras correcciones

Detalle de calidad

Figura 13. Resultados del análisis de calidad de código con SonarCloud.

Como resultado del análisis realizado con SonarCloud, el proyecto obtuvo un Quality Gate aprobado. Asimismo, alcanzó una cobertura de pruebas del 81,9 %, sin bugs ni vulnerabilidades detectadas. Se identificaron 11 code smells y un 8,4 % de líneas duplicadas, indicadores considerados para las acciones de mejora del código.



Figura 14. Indicadores de calidad y cobertura del proyecto MediLink mostrados en el README de GitHub.

Nota. La imagen presenta el estado del Quality Gate, la cobertura de pruebas, los bugs, las vulnerabilidades, los code smells y el porcentaje de líneas duplicadas.

Gestión de incidencias con Jira Service Management

Las observaciones relevantes fueron registradas como incidencias para asignar responsables y realizar seguimiento hasta su corrección.

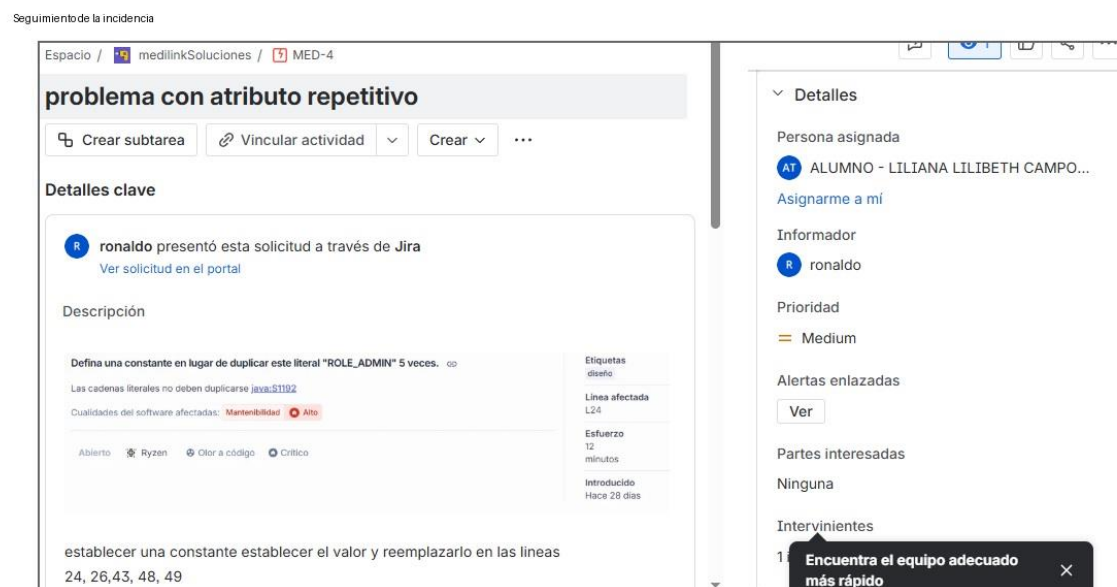
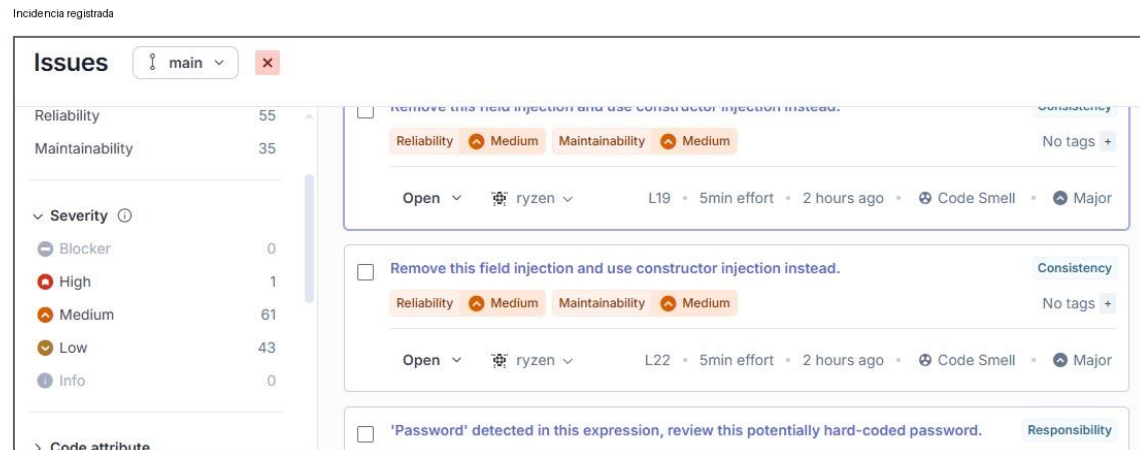


Figura 15. Registro y seguimiento de incidencias mediante Jira Service Management.

Se integró SonarCloud dentro del flujo de GitHub Actions para evaluar automáticamente la calidad del código, detectar bugs, vulnerabilidades y duplicidad. El análisis se ejecuta en cada push a main y develop, y en cada Pull Request hacia main, generando un reporte que el equipo revisa antes de aprobar la fusión de cambios. Las incidencias detectadas (bugs, code smells y vulnerabilidades) fueron reportadas al equipo mediante Jira Service Management para su seguimiento y corrección.

Cobertura de Pruebas con JaCoCo

Reporte de cobertura

JaCoCo generó un reporte de cobertura por paquetes y clases, permitiendo reconocer qué partes del código fueron ejecutadas por las pruebas automatizadas y cuáles requieren pruebas adicionales.

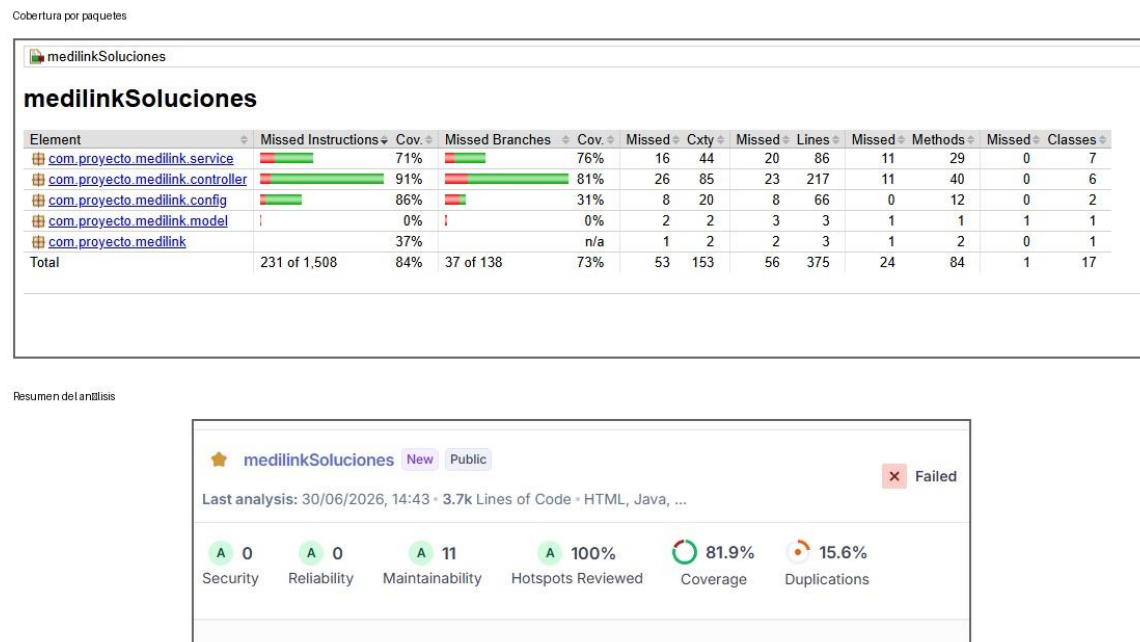


Figura 16. Reporte de cobertura de pruebas generado con JaCoCo.

Luego de corregir los problemas críticos identificados por SonarCloud (seguridad, redundancia y duplicidad), se generó un reporte de cobertura de pruebas con JaCoCo, integrado en el mismo flujo de GitHub Actions, lo que permite visualizar qué porcentaje del código está cubierto por pruebas automatizadas.

Contenedores con Docker

Construcción de la imagen Docker

La aplicación fue empaquetada mediante una imagen Docker. Se verificó el proceso de construcción y la creación correcta de la imagen antes de ejecutar los servicios.

Construcción de la imagen

```
C:\Users\USER>cd C:\Users\USER\Downloads\cursos 2026\herramientas de desarrollo\semana 14\Medilink

C:\Users\USER\Downloads\cursos 2026\herramientas de desarrollo\semana 14\Medilink>fir
"fir" no se reconoce como un comando interno o externo,
programa o archivo por lotes ejecutable.

C:\Users\USER\Downloads\cursos 2026\herramientas de desarrollo\semana 14\Medilink>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 089D-CC05

Directorio de C:\Users\USER\Downloads\cursos 2026\herramientas de desarrollo\semana 14\Medilink
26/06/2026  18:20  <DIR>          .
26/06/2026  18:20  <DIR>          ..
24/06/2026  10:38                40  .gitattributes
24/06/2026  10:39  <DIR>          .github
24/06/2026  10:38                1,507 .gitignore
24/06/2026  10:38  <DIR>          .mvn
24/06/2026  11:28  <DIR>          .vscode
24/06/2026  21:20                707 Dockerfile
24/06/2026  10:38  <DIR>          docs
24/06/2026  10:38                10,001 FAQ_SWAGGER.md
24/06/2026  10:38                1,768 HELP.md
24/06/2026  10:38  <DIR>          informes
24/06/2026  10:38                11,558 LICENSE
24/06/2026  10:38                11,790 mvnw
24/06/2026  10:38                8,481 mvnw.cmd
24/06/2026  10:38                3,599 pom.xml
24/06/2026  10:38                9,057 README.md
24/06/2026  10:38  <DIR>          resultados
[+] Building 191.0s (9/16)                                docker:desktop-linux
=> [build 1/6] FROM docker.io/library/maven:3.9.9-eclipse-temurin-17@sha256:f58d59b6273e785ac0a4477f6e9b5baid7 183.1s
=> => sha256:2d56ad71dc3b91726c3f2ceaa3a5917bddafd133435af8f5b444a4f3e690492b 359B / 359B 0.6s
=> => sha256:3fa0bcda0f1da02f4ec2a7978b9d08b534ab12742726582ba3908e70f0329ffc 848B / 848B 0.4s
=> => sha256:e778c13590fd085ade1b77b7188d40026e1c681d1b51f982519a3cc79faa4c5 9.17MB / 9.17MB 22.6s
=> => sha256:ddcf5fb2d06ceba3ca7015c4e844c4ff884ef85f4b22be6bde29e1c3a9b5be67 22.54MB / 22.54MB 48.4s
=> => sha256:c70eff9bfbfbfe89f4317fcb629c8b55f368a6376f739f347d783caf8d2fe79 160B / 160B 1.1s
=> => sha256:d9815e77b54f33dbcd213d596eeaad209e5255afa9c22666ba991c8c1daf1211 2.28kB / 2.28kB 0.8s
=> => sha256:7bbafedfaf521e89144400b08c3dbacde18a658508d1b700f2b98bf123959ebe 22.95MB / 22.95MB 38.7s
[+] Building 462.8s (17/17) FINISHED                                docker:desktop-linux
```

Imagen generada

```
C:\Users\USER\Downloads\pruebas\Nueva carpeta\Medilink>docker images

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
medilink-app:latest  48cc3138b962      504MB         155MB
```

Figura 17. Construcción y verificación de la imagen Docker de MediLink.

Ejecución con Docker Compose

Docker Compose permitió iniciar de manera conjunta la aplicación y la base de datos MySQL. Posteriormente, se verificó que ambos contenedores se encontraran activos.

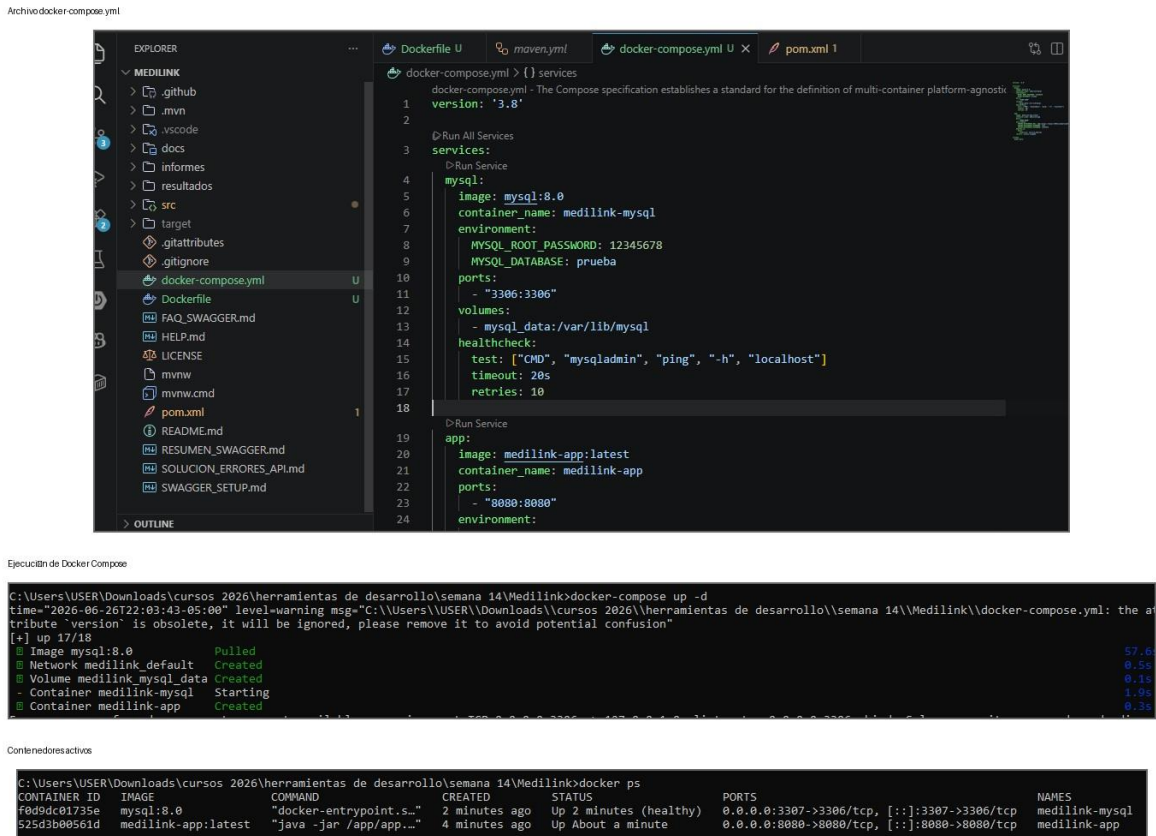
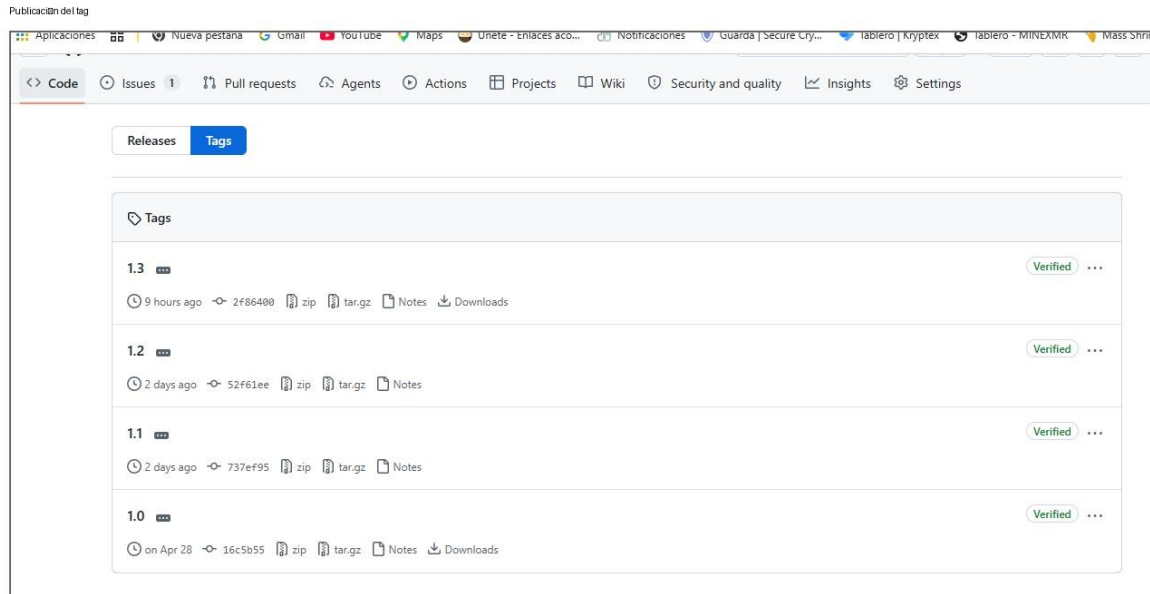


Figura 18. Configuración y ejecución de los contenedores mediante Docker Compose.

Para empaquetar la aplicación se creó un Dockerfile multi-etapa: una primera etapa compila el proyecto con Maven y una segunda etapa, más ligera, ejecuta la aplicación sobre un JRE, optimizando el peso final de la imagen. Adicionalmente, se creó un archivo docker-compose.yml que integra dos servicios: la base de datos MySQL y la aplicación MediLink, permitiendo levantar todo el entorno con un solo comando en cualquier máquina.

Los comandos principales utilizados fueron: `docker build -t medilink-app .` para construir la imagen; `docker-compose up -d` para levantar los contenedores en segundo plano; `docker ps` para verificar que los contenedores estén activos; y `docker stop` / `docker start` para detener e inicializar nuevamente los servicios.



Release del proyecto



Figura 20. Creación del tag y publicación del Release del proyecto.

Finalmente, la aplicación fue desplegada utilizando Railway, una plataforma en la nube (PaaS) que permite desplegar aplicaciones, bases de datos y servicios de forma rápida, sin necesidad de configurar servidores manualmente ni lidiar con la complejidad de proveedores como AWS o Azure. El proyecto se encuentra actualmente disponible en la siguiente URL:

medilink-production-9950.up.railway.app

credencial de administrador

admin@admin.com

admin123

Resultados

Como resultado del desarrollo del proyecto MediLink, se logró implementar y desplegar una plataforma web funcional para la gestión de citas y consultas médicas, cumpliendo con los objetivos planteados en los distintos hitos de entrega:

- Se desarrolló un backend en Spring Boot con arquitectura en capas (controlador, servicio, repositorio) que soporta la autenticación de usuarios mediante JWT y el control de roles (administrador, médico, paciente).
- Se implementó el módulo de gestión de citas médicas, permitiendo el registro, la consulta, la cancelación y la validación de disponibilidad de horarios por especialidad.
- Se configuró un flujo de integración continua (CI) en GitHub Actions que ejecuta automáticamente las pruebas del proyecto en cada push, logrando compilaciones exitosas y consistentes.
- Se integró SonarCloud al flujo de CI, lo que permitió identificar y corregir bugs, vulnerabilidades y código duplicado, mejorando de forma medible la calidad del código a lo largo del proyecto.
- Se empaquetó la aplicación en un contenedor Docker multi-etapa junto a su base de datos MySQL, verificando su correcto funcionamiento local mediante docker-compose.

- Se desplegó la aplicación en producción utilizando Railway, obteniendo una URL pública operativa: <https://medilink-production-8cf3.up.railway.app/>.
- Se mantuvo un flujo de trabajo colaborativo ordenado, con ramas protegidas, revisiones obligatorias de código y un tablero Kanban en GitHub Projects vinculado a los Issues del repositorio.

Conclusiones

- La arquitectura basada en Spring Boot permitió construir un backend modular y escalable, facilitando la incorporación de nuevos módulos (citas, especialidades) sin afectar la estabilidad del sistema, validando así la hipótesis específica HE1.
- La adopción de la metodología Scrum, apoyada en GitHub Projects, favoreció una entrega incremental ordenada del producto y una comunicación constante dentro del equipo de trabajo.
- La incorporación de Spring Security y JWT permitió cumplir con los requerimientos de seguridad planteados en el objetivo OE3, protegiendo la confidencialidad e integridad de los datos clínicos.
- La automatización mediante GitHub Actions, SonarCloud y Docker redujo el margen de error humano en el proceso de integración y despliegue, y facilitó detectar problemas de calidad de forma temprana.
- El despliegue en Railway demostró que es posible llevar el proyecto a un entorno de producción real sin una infraestructura compleja, cumpliendo con el objetivo general del proyecto.

Recomendaciones

- Completar la implementación del módulo de videollamadas mediante WebRTC, que a la fecha de este avance se encuentra pendiente de integración.
- Ampliar la cobertura de pruebas automatizadas reportada por JaCoCo antes de la entrega final del proyecto.
- Documentar el uso de GitHub Secrets para el manejo de credenciales en el entorno de producción, evitando exponer contraseñas o tokens en el código fuente.
- Generar el Release oficial v1.0.0 en GitHub, adjuntando el código fuente y el archivo .jar compilado como cierre formal del ciclo de desarrollo.

Referencias

- Beas, D. (2014). El mundo que viene en la “segunda era de las máquinas”. *Política Exterior*, 28(161), 170–174. <https://dialnet.unirioja.es/servlet/articulo?codigo=4810962>
- Instituto Nacional de Estadística e Informática. (2024). *Aumentó la población usuaria de internet en todos los grupos de edad en el primer trimestre de 2024*. <https://m.inei.gob.pe/prensa/noticias/aumento-la-poblacion-usuaria-de-internet-en-todos-los-grupos-de-edad-en-el-primer-trimestre-de-2024-15248/>
- Orosco, J., Carrión, K., & Salinas, N. (2021). Desarrollo de una aplicación web de telemedicina en beneficio de la comunidad de la Universidad Técnica Estatal de Quevedo. *Ingeniería e Innovación*, 8(22). <https://doi.org/10.21897/23460466.2328>
- Padilla, C., Avellaneda, J., & Medina, R. (2024). *Implementación de software para citas médicas a través de canales virtuales* [Tesis de maestría, Universidad Peruana de Ciencias Aplicadas]. Repositorio Académico UPC. <http://hdl.handle.net/10757/684115>
- Universidad de Cataluña. (s. f.). *Seguridad informática: La importancia y lo que debe saber*. <https://www.ucatalunya.edu.co/blog/seguridad-informatica-la-importancia-y-lo-que-debe-saber>